ENGINEERING • AUTONOMOUS SYSTEMS

WORK BREAKDOWN STRUCTURE (WBS)

& Project Execution Plan

Cube Attitude Control and Stabilization System
via Reaction Wheels and Computer Vision

MEETING DATE
October 16

Thursdays

MONITORING

PLANNING • DEVELOPMENT • INTEGRATION • VALIDATION

1. Team Constitution and Engineering Profiles

Name	Engineering / Field	Technical Profile and Focus Area
Ana	Aerospace (Profile 1)	Attitude Simulation and Dynamics (MSYS2, Roll/Pitch damping, Yaw control).
Maurício	Aerospace (Profile 2)	Control Architecture (Cascaded models: Attitude → Angular Vel. → Wheel Vel.).
Ricardo	Aerospace (Profile 3)	Communication Protocol and ICD (Interface Control Document TMTC).
Edson	Electrical and Computer	Firmware and Hardware with **FreeRTOS** (ESP-IDF, I2C IMU, CAN-FD, Device Drivers).
Alessia	Industrial Automation	Ground Station (Python, PyQt5, pyqtgraph, Telemetry and GUI).
Professor	General Guidance	Teaching and External Validation of Milestones and Development Levels.

2. Hardware and Components Overview

The project is based on a time-sensitive *Embedded System*, focused on the control and stabilization of a cube. The hardware and base system architecture includes:

- **System on a Chip (SoC):** A multi-core main microcontroller (CPU) operating on a Real-Time Operating System (**FreeRTOS**) — **abandoning the *Baremetal* approach** — to ensure that the critical task of balancing the cube has top priority.
- **Connectivity:** The chip integrates Wi-Fi and Bluetooth (BLE) communication capabilities.
-
- **Power and Safety:** A battery powers the devices and motors, coupled with an electronic voltage measurement system and a *buzzer* to warn against deep discharge.
- **Navigation Sensors:** An IMU (*Inertial Measurement Unit*) and a magnetometer that provide gravity, frequency, and acceleration data.

- **Actuators (Motors):** Utilization of three reaction wheels, communicating with their own controllers.
- **Camera:** Connected via SPI. Resolution must be kept to a minimum (*grayscale*) so as not to overload the processor during target identification.

3. Communication Protocols and Interfaces

Communication requires the rigorous definition of protocols (**the UART protocol was strictly excluded from the project**):

- **SPI and I2C:** Synchronous protocols for internal communication. Sensors (IMU/Magnetometer) via I2C; Camera and Transceivers via SPI. The team must document latencies, MTU, transmission speeds, duplex type, *master-slave/multi-slave* architecture, and the respective *Chip Select*.
- **CAN-FD:** Motors communicate via CAN-FD (not to be confused with standard CAN).
- **SPI to CAN-FD Converter:** Due to the lack of suitable ports on the microcontroller, an intermediate chip will be used to convert the SPI protocol into CAN-FD, ensuring the delivery of *setpoints* to the motors.
- **TMTC (Telemetry and Telecommand):** Protocol to communicate with the *Ground Station*, established over Wi-Fi (UDP), with the possibility of implementing **redundancy via BLE**. Requires rigorous checks (CRC), *acknowledgements*, and defined commands (e.g., 90° rotation, up/down, *Emergency Stop*, *Pings*).

4. Software Architecture Strategy (Multicore Processing)

To ensure system stability, the team strongly recommended dividing tasks between the two microcontroller *cores*:

- **Core 1 (Critical Control):** Dedicated exclusively to the mechanical stabilization and control of the cube. This core constantly reads the IMU, checks *setpoints*, and adjusts the motors to ensure the cube does not lose control. The golden rule is that the cube must maintain its position and stay stabilized even if the Wi-Fi connection goes down.
- **Core 2 (Acquisition and Communication):** Dedicated to camera image acquisition, data processing (such as identifying a white target point), and managing Wi-Fi communication (TMTC) with the *Ground Station*.
- **Image Processing (Onboard vs. Ground Station):** Although it is possible to send the complete image via Wi-Fi to be processed on the *Ground Station*, it was highly recommended that image processing (searching for a white dot on a black screen) be performed *onboard* (inside the cube itself). This saves bandwidth and allows the system to make orientation decisions autonomously.

5. Project Implementation Phases

Project execution must follow a step-by-step approach (*"eating the elephant one bite at a time"*), advancing progressively:

1. **Phase 1: Requirements Analysis and Design:**
    - Understand components, read *datasheets*, and figure out how to interact with devices and what data to extract.
    - Define communication protocols and create the ICD (*Interface Control Document*), which documents all software interfaces.
    - Define *Device Drivers* (hardware abstraction layers) and software architecture.
2. **Phase 2: IMU Data Acquisition:**
    - Program the device to read the IMU and magnetometer.

- Send this data via Wi-Fi to the *Ground Station* and visualize it in real-time charts. When moving the cube physically, the charts on the station must react in real time.

3. **Phase 3: Basic Motor Control:**
   - Create commands on the *Ground Station* to send specific speed or torque to the motors.
   - Ensure the reaction wheels start spinning according to the command and receive feedback (telemetry) from the motor controller.
4. **Phase 4: Autonomous Position Control:**
   - Define a position *setpoint* on the *Ground Station* (e.g., "rotate 90 degrees").
   - The cube's software must cross-reference IMU data with the motors to rotate automatically and remain fixed in that position continuously.
5. **Phase 5: Image Acquisition:**
   - Connect the camera to the system, acquire images, and display them on the *Ground Station*.
6. **Phase 6: Target Tracking:**
   - Process the image to identify a white spot against a black background.
   - Cross-reference this point's position with motor control so that the cube moves and "follows" or points to the target autonomously.

6. Simulation Strategies and Testing Methodology

- **Mandatory Simulation:** C/C++ code (*firmware*) must be tested with the *Ground Station* (Python) in a local simulated environment before uploading to the *hardware*.
- **Dummy Data Generation:** Provisory implementation of arbitrary signals to test GS robustness, parsing, and error correction (CRC).

7. Detailed Work Breakdown Structure (WBS)

The following matrix details the work breakdown structure, organized by Work Packages (WP). The project will operate with an **internal deadline of November 30** for the completion of technical development. The subsequent week constitutes a ***buffer zone*** dedicated exclusively to code improvement, documentation consolidation, and delivery preparation.

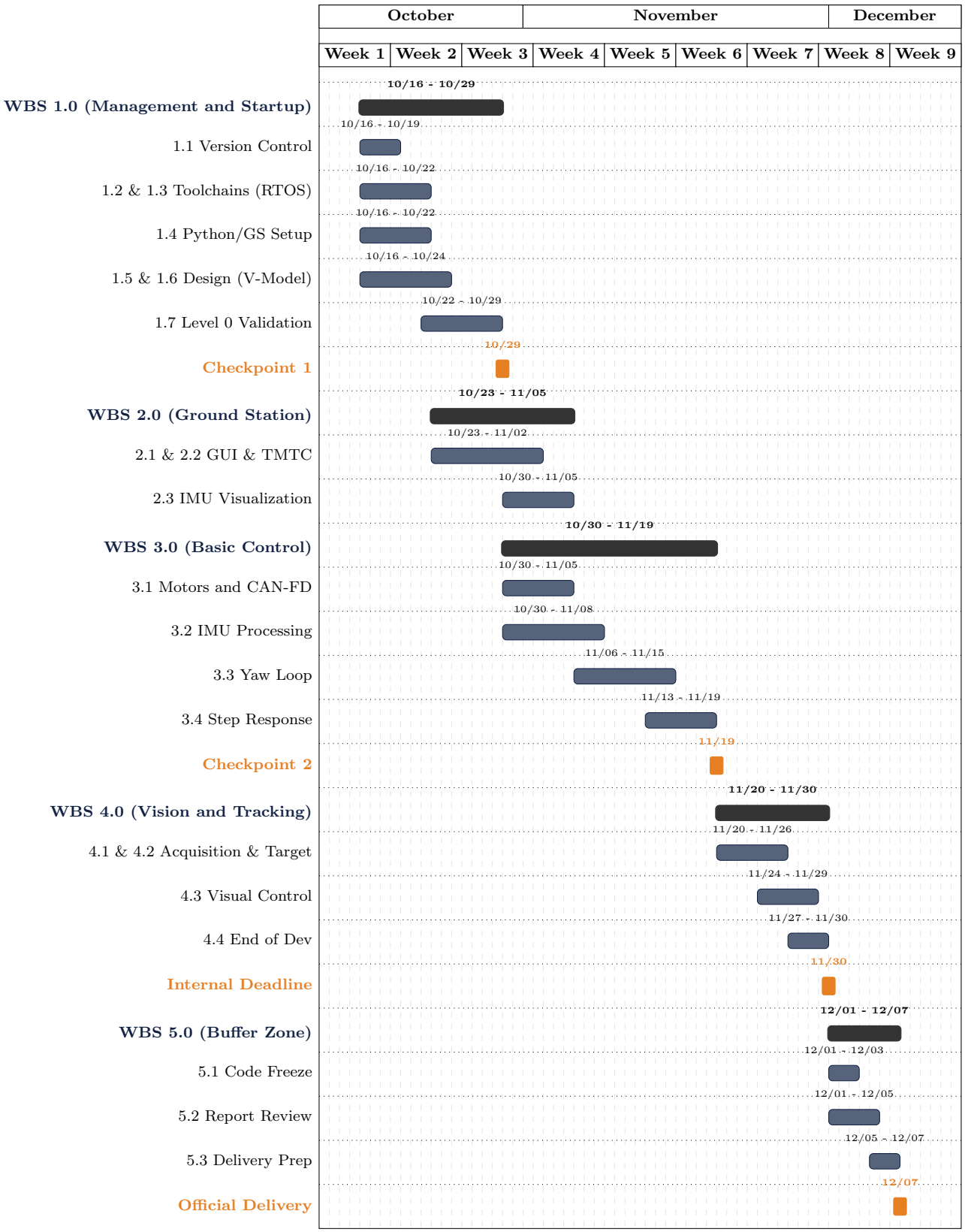
Table 1: Integrated Matrix of Tasks, Responsibles, Dates, and WBS Deliverables

Code	Work Package	Technical Tasks Description	Responsible	Temp. Window	Milestones
WBS 1.0 — Management, Environment Setup & Requirements (Level 0)					
1.1	Version Control	GIT installation and *commit* documentation.	Entire Team	Oct 16 – Oct 19	Kick-off
1.2	MSYS2 Environment	C/C++ toolchain setup in MSYS2 for simulation.	Ana	Oct 16 – Oct 22	Level 0
1.3	ESP-IDF Setup	VS Code installation, ESP-IDF extension, and FreeRTOS.	Edson	Oct 16 – Oct 22	Level 0
1.4	Python / GS Setup	Installation of Python, *numpy*, *PyQt5*, and *pyqtgraph*.	Alessia	Oct 16 – Oct 22	Level 0
1.5	TMTC ICD Protocol	Technical drafting of packets, Wi-Fi UDP, BLE, and CRC.	Ricardo	Oct 16 – Oct 24	Level 0

Code	Work Package	Technical Tasks Description	Responsible	Temp. Window	Milestones
1.6	Control Design	Cascaded control modeling and dynamics simulator.	Maurício	Oct 16 – Oct 24	Level 0
1.7	Level 0 Validation	Compilation of base project and sending of *Dummy Data*.	Entire Team	Oct 22 – Oct 29	**CP1: Oct 29**
WBS 2.0 — Ground Station, Interface & Simulated Communication					
2.1	Ground Station GUI	Creation of graphic *mockup* and data windows.	Alessia	Oct 23 – Nov 02	Level 1
2.2	TMTC Emulator	Local simulation of Cube/GS network communication.	Ricardo	Oct 23 – Nov 02	Level 1
2.3	IMU Visualization	Real-time plotting on GS of IMU/Magnetometer data.	Alessia/Edson	Oct 30 – Nov 05	Level 2
WBS 3.0 — Basic Control, Stabilization, and Attitude (Levels 1 to 3)					
3.1	Motor Driver / CAN	SPI/CAN-FD implementation and motor control (< 0.5s).	Edson	Oct 30 – Nov 05	Level 1
3.2	IMU Processing	I2C IMU reading and noise filtering (error $< 2°$).	Ana/Edson	Oct 30 – Nov 08	Level 2
3.3	Yaw Control Loop	Autonomous control on Yaw axis and Roll/Pitch damping.	Maurício/Ana	Nov 06 – Nov 15	Level 3
3.4	Step Response	Autonomous control test (45° *step*, error $\pm 1°$).	Maurício/Team	Nov 13 – Nov 19	**CP2: Nov 19**
WBS 4.0 — Computer Vision, Onboard Processing & Tracking					
4.1	Image Acquisition	SPI camera integration and *core* priorities.	Edson/Alessia	Nov 20 – Nov 24	Level 4
4.2	Target Processing	*Onboard* target detection algorithm (≥ 15 fps).	Ricardo/Edson	Nov 20 – Nov 26	Level 4
4.3	Visual Control	Connection of visual error with motors (centering in < 6s).	Maurício/Ana	Nov 24 – Nov 29	Level 5
4.4	Real Cube Tests	Technical conclusion and final physical tests **(End of Dev)**.	Entire Team	Nov 27 – Nov 30	**DL: Nov 30**
WBS 5.0 — Buffer Zone: Review, Documentation, and Final Delivery					
5.1	*Code Freeze*	Code improvement (cleanup, comments) and GIT consolidation.	Entire Team	Dec 01 – Dec 03	Review
5.2	Report Review	Final review of chapters, formatting, and V-Model verification.	Entire Team	Dec 01 – Dec 05	Review
5.3	Delivery Prep	Prepare final demonstration for defense and file submission.	Entire Team	Dec 05 – Dec 07	**Deliv.: Dec 07**

8. Project Schedule (Gantt Chart)

Below is the chronological mapping of the WBS structure. Activities are grouped by main work packages, aligned with the **internal deadline set for November 30** for development completion. The last week acts as a ***buffer zone***.


October
November
December
Week 1
Week 2
Week 3
Week 4
Week 5
Week 6
Week 7
Week 8
Week 9
WBS 1.0 (Management and Startup)
10/16 - 10/29
1.1 Version Control
10/16 - 10/19
1.2 & 1.3 Toolchains (RTOS)
10/16 - 10/22
1.4 Python/GS Setup
10/16 - 10/22
1.5 & 1.6 Design (V-Model)
10/16 - 10/24
1.7 Level 0 Validation
10/22 - 10/29
Checkpoint 1
10/29
WBS 2.0 (Ground Station)
10/23 - 11/05
2.1 & 2.2 GUI & TMTC
10/23 - 11/02
2.3 IMU Visualization
10/30 - 11/05
WBS 3.0 (Basic Control)
10/30 - 11/19
3.1 Motors and CAN-FD
10/30 - 11/05
3.2 IMU Processing
10/30 - 11/08
3.3 Yaw Loop
11/06 - 11/15
3.4 Step Response
11/13 - 11/19
Checkpoint 2
11/19
WBS 4.0 (Vision and Tracking)
11/20 - 11/30
4.1 & 4.2 Acquisition & Target
11/20 - 11/26
4.3 Visual Control
11/24 - 11/29
4.4 End of Dev
11/27 - 11/30
Internal Deadline
11/30
WBS 5.0 (Buffer Zone)
12/01 - 12/07
5.1 Code Freeze
12/01 - 12/03
5.2 Report Review
12/01 - 12/05
5.3 Delivery Prep
12/05 - 12/07
Official Delivery
12/07

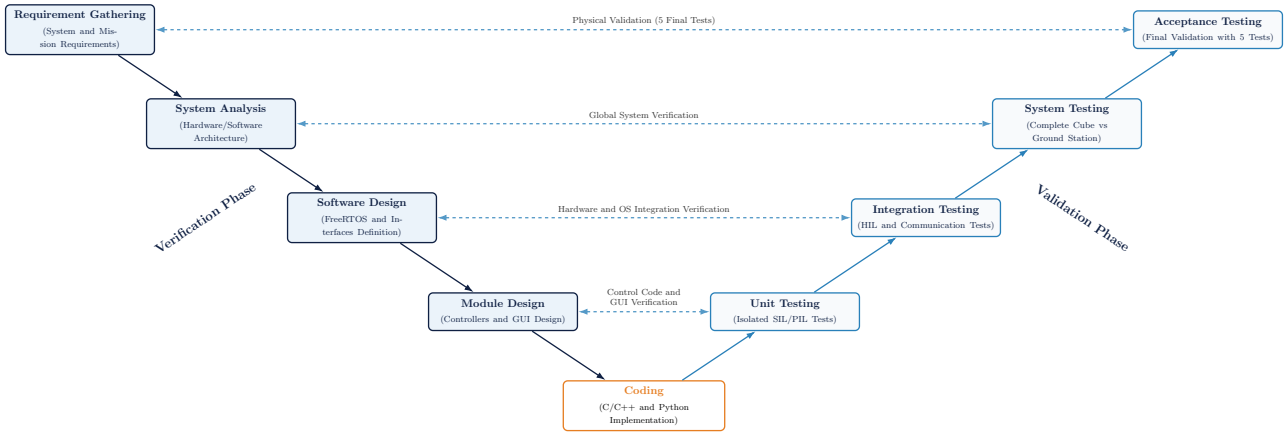
8.1 Strategic Schedule, Milestones, and Deliverables - SUMMARY

Date / Event	Monitoring	Main Objective and Success Criteria
October 16	Initial Meeting	Kick-off, profile allocation, and start of WBS based on the V-Model.
October 22	Class (Thursday)	Validation of local environments (MSYS2, ESP-IDF, Python) and ICD draft.
October 29	**CHECKPOINT 1**	**Level 0 Completed:** Compilation and simulation of UDP telemetry *Dummy Data*.
November 05	Class (Thursday)	Python GUI demonstration responding to real-time data and CAN-FD/I2C tests.
November 12	Class (Thursday)	Validation of attitude estimation (IMU) and wheel speed control.
November 19	**CHECKPOINT 2**	**Levels 1 to 3 Completed:** Autonomous position control (45° step response, error $\leq \pm 1°$).
November 26	Class (Thursday)	Presentation of *onboard* image processing and target point detection.
November 30	**INTERNAL DEADLINE**	**End of Development:** Technical conclusion and physical tests. Start of *Buffer Zone*.
December 03	Class (*Buffer Zone*)	**Code Freeze and Review:** Code cleanup, GIT consolidation, and final report review.
December 07	**FINAL DELIVERY**	**Official Submission of Project Report, GitHub Repository, and Demonstration.**

9. V-Model Development Methodology

The execution of this project strictly adopts the V-Model, ensuring that development, design, and architecture are directly coupled to a progressive testing and validation phase. The model is divided into three main axes: the Verification Phase (descending), the Coding Phase (base), and the Validation Phase (ascending).

Given the critical nature of embedded attitude control systems, pure programming on the physical processor only occurs after the strict definition of interfaces (left arm of the V) and integration is carried out continuously and tested at each equivalent level (right arm of the V).



9.1. Strategic Mapping and Phase Coupling (Project Applied)

The true strength of the V-Model lies in the direct coupling between abstract design phases (left arm) and their respective test and verification phases (right arm). To ensure a rigorous reading of our attitude control system lifecycle, the development of the cube is detailed below in accordance with the horizontal arrows of the diagram, uniting each level of abstraction to its corresponding validation and the tasks designated in the **Work Breakdown Structure (WBS)**.

Level 1: Physical Validation and Mission

Macro Coupling: *Requirement Gathering* $\longleftrightarrow$ *Acceptance Testing*

- **Requirement Gathering (What the system must be):** This is the foundational stage of the project. The primary objective of stabilizing the cube using reaction wheels and computer vision is defined. Non-negotiable hardware constraints are identified: the imperative use of an ESP32 *dual-core* microcontroller, critical weight and power (battery) management, and the need for a low-resolution grayscale camera so as not to exhaust the CPU. The UART protocol is categorically rejected in favor of SPI, I2C, and CAN-FD.

- **Acceptance Testing (The proof of success upon delivery):** This is the culmination of the cycle (Level 6). The final system is evaluated against the requirements demanded at the outset through **5 flawless physical tests**:

    1. **Acquisition:** Drop the cube with a 30° deviation and force centering in less than 6 seconds.
    2. **Hold:** Maintain the system in steady-state, supporting a maximum deviation of ±2° for 1 minute.
    3. **Disturbance:** Apply a physical push ($\sim$ 20°) and evaluate autonomous centering in less than 4 seconds.
    4. **Dynamic Tracking:** Slow movement of a reference placard while the cube tracks it autonomously.
    5. **Visual Occlusion:** Abruptly cover the camera for 2 seconds; sensor fusion (IMU) must ensure the cube does not lose its spatial orientation.

WBS Framing

¡small¿ **WBS 1.5 (TMTC ICD Protocol):** Strict establishment of mission communication rules and telemetry.
WBS 4.4 (Real Cube Tests): Execution of final physical tests for global device acceptance.
WBS 5.3 (Delivery Prep): Final demonstration with all parameters validated.

Level 2: Global System Verification

Architectural Coupling: *System Analysis* $\longleftrightarrow$ *System Testing*

- **System Analysis (Physical and Logical Division):** Mapping of the communication infrastructure and component interaction. It is decided to separate the system between the physical Cube and the Ground Station (PC), interconnected via Wi-Fi (UDP) with the possibility of BLE redundancy. The sensor network topology (IMU and Magnetometer) is established via I2C. The architectural need to incorporate a physical SPI to CAN-FD converter chip to make up for the lack of ports on the ESP32 is also defined, allowing the delivery of *setpoints* to the motors.
- **System Testing (Assembled Cube Evaluation):** The chassis is fully enclosed with all electronic components and battery on board. Systemic response tests to a 45° step are performed, requiring a steady-state error of $\leq \pm 1°$. Critical network failure simulations are forced: the cube must recognize the drop of the Wi-Fi connection to the Ground Station and, completely isolated, not collapse, keeping motor stabilization active uninterruptedly.

WBS Framing

WBS 1.6 (Control Design): Modeling of overall cascaded system dynamics.
WBS 3.4 (Step Response): Testing of the closed-system autonomous control loop (45° *step*).
WBS 4.3 (Visual Control): Final coupling of the image positional error (camera) to the motors.

Level 3: Hardware and OS Integration Verification

Infrastructure Coupling: *Software Design* $\longleftrightarrow$ *Integration Testing*

- **Software Design (FreeRTOS Architecture and ICD):** Definitively abandoning the *baremetal* structural approach, the use of **FreeRTOS** is established. High-level firmware is divided into a restricted multicore allocation model: **Core 1** is dedicated exclusively and with maximum criticality to the motor control loop and IMU reading; **Core 2** is in charge of camera acquisition and TMTC communication. The first version of the ICD (*Interface Control Document*) is drafted, documenting the structure of all packets, CRC validations, and telemetry commands.
- **Integration Testing (Bench HIL Test):** Known as HIL (*Hardware-in-the-loop*) testing. Connection between breadboards begins, linking the real processor (ESP32) to motors, actuators, and navigation sensors. This phase rigorously proves the viability of *Software Design*: it proves that **Core 2** is capable of sending heavy images and telemetry via UDP without causing jolts, stalls, or latencies that collapse the critical stabilization cycle of **Core 1**.

WBS Framing

WBS 1.3 (ESP-IDF Setup): Integration and base configuration of FreeRTOS and microcontroller compilation.
WBS 3.1 (Motor Driver/CAN-FD): Connection of reaction wheels and peripheral protocols.
WBS 4.1 (Image Acquisition): Connecting the SPI camera and testing *multicore* scheduling stability.

Level 4: Control Code and GUI Verification

Functional Coupling: *Module Design* $\longleftrightarrow$ *Unit Testing*

- **Module Design (Logical and Mathematical Design):** Focus on micro-planning of analytical and visual functions. Control architecture is modeled in the classic error cascade format (Visual Error $\rightarrow$ Attitude $\rightarrow$ Angular Vel. $\rightarrow$ Wheel Vel.). Formulas for raw noise filtering for the IMU are defined, and the mathematical algorithm to identify, directly onboard the cube, a white target point on a dark screen is conceptualized. In parallel, the visual and architectural mockup of the Ground Station GUI supported in Python, *PyQt5*, and *pyqtgraph* is created.
- **Unit Testing (Isolated SIL/PIL Tests):** Where mathematical and logical modules are exposed to closed test batteries (Unitary) without needing the complete equipment, corresponding to SIL (*Software-in-the-loop*) and PIL (*Processor-in-the-loop*) validations. It is validated whether the filter designed for IMU reading (I2C) actively limits accumulated error to $< 2°$. It is ensured with test images that the algorithm properly finds the matrix white target, and CAN-FD frame generation is tested individually in the logical simulator.

WBS Framing

WBS 2.1 & 2.2 (GUI and TMTC Emulator): Programming and unit testing of visual windows and simulated packets.
WBS 3.2 (IMU Processing): Isolated creation of noise attenuation and angular calculation algorithms.
WBS 4.2 (Target Processing): Validation of the black screen vs white pixel matrix scanning algorithm.

The Central Vertex: Base Coding (*Coding*)

The tipping point: Where Design abstractions convert into tangible C/C++ code.

- **Technical Production and Simulator:** This is the moment where all abstraction from the left arm materializes. Exhaustive implementation of the C/C++ *firmware* begins, taking advantage of the ESP-IDF toolchain installed in the MSYS2 environment, in tune with writing the Python graphical interface for the ground station. A vital guideline of this project indicates that half of the work is advanced directly through programming in a **simulated environment**, generating the cube's physics on a computer and forcing the exchange of *dummy data* to accelerate the detection of chronic errors and bugs in the algorithm, long before burning the code onto the sensitive physical *hardware*.

WBS Framing

WBS 1.2 & 1.4 (MSYS2 and Python Environment): Definition and calibration of programming languages across the team.
WBS 1.7 (Level 0 Validation): Primary code translation and initial verification of *dummy data* in emulator.
WBS 3.3 (Yaw Control Loop): Massive writing of mathematical code in C/C++ for attitude and motor control.